# Assessment Brief

| | |
|---|---|
| **Module code:** | WM9SL-15 |
| **Module title:** | AI and Games |
| **Assessment title:** | Game AI Implementation |
| **Weight:** | 100% |
| **Length:** | 4000 words (report component) |
| **Submission format:** | Project – Individual |
| **Submission deadline:** | 31st May 2026 |
| **Self-certification eligible:** | Yes (extension only) |

## Overview

You will architect and implement artificial intelligence techniques in C++ using the game engineering shell (game loop, renderer, input, basic drawing) provided into a single interactive system within a modern development environment then write a 4000-word technical report documenting your design decisions, implementation, and critical evaluation.

The project is designed to demonstrate both your practical ability to implement game AI systems and your capacity to analyse and justify the approaches you have chosen. Code alone is not sufficient evidence of learning: the report is the primary vehicle through which you demonstrate achievement of the module learning outcomes.

## Requirements

Your submission must address the features in sections A, B and C below.

### A. Traditional Game AI

Every submission must include a working implementation of each of the following: - A pathfinding system using A* or an equivalent informed search algorithm on a graph representation suitable for your environment (tile grid, waypoint graph, or navigation mesh) - A decision system for at least one agent using a finite state machine, behaviour tree, or equivalent hierarchical decision structure - A spatial sensing or perception component (including range detection, line-of-sight, field-of-view sensor) that can feed into an agent decision system

Your Choice of One of the following:

1. Hierarchical pathfinding (cluster abstraction or equivalent) with measurable improvement in query time or path quality
2. Behaviour tree with blackboard architecture, reusable subtrees, and decorator node types
3. Utility AI scoring system with response curves applied to at least two competing agent objectives

4. Goal-Oriented Action Planning (GOAP) or STRIPS-style planner producing runtime action sequences for at least one agent

More marks will be awarded for the more technically accomplished implementations from this list.

### B. Neural Architecture for Game Intelligence

Apply a neural network to a game-relevant predictive or control task, with the emphasis on architectural justification: you must argue why the chosen architecture's inductive bias fits the structure of your specific problem, and compare results against a simpler baseline. The architecture must be one of: convolutional (CNN), recurrent (RNN/LSTM), or transformer/attention. Choose the approach **Supervised Learning** or **Reinforcement Learning** that fits your project:

1. *Supervised Learning:* Define a labelled dataset from your game (e.g. classify game states, predict player intent, estimate threat from sensor readings). Train and evaluate on held-out test data. Required evidence: dataset description, training loss curve, test accuracy or equivalent metric, baseline comparison.
2. *Reinforcement Learning:* Train a policy network by interacting with your game environment through a reward signal, no labelled data required. Required evidence: reward function design rationale, learning curve (cumulative reward over training steps), policy analysis describing what the agent learned and where it fails, baseline comparison.

In both cases the report must include an architecture diagram and a written argument for why the chosen architecture is appropriate for the problem.

### C. Neural Generative Model for Game Content

Use a generative model to produce novel game content – sprites, textures, tile sets, levels, dialogue, or audio. Suitable architectures include GANs, VAEs, diffusion models, and autoregressive transformers. Training may be performed offline in Python; the trained model must be integrated into or demonstrated alongside your game environment. The generated content should be integrated into your runtime environment from Section B.

The emphasis is on evaluating output quality rather than architectural justification. You must define a quality metric appropriate to your content type and compare generated output against a baseline (random generation, rule-based generation, or human-authored examples). Required evidence: generated output samples, quality metric definition and rationale, baseline comparison results.

## Deliverables

Runtime in C++ on top of the Games Engineering Shell Provided. Combining the work from each section (Traditional, Neural Architecture and Neural Generative) into a single functioning game experience.

Where necessary you may use Python to manage the training and data access.

During runtime consideration should be given to keep the speed of inference high, you should describe and evaluate the approach you took.

### Deliverable 1: Working System and Code

A working implementation of AI techniques addressing all the requirements (A, B, & C) within a game or interactive environment. The codebase and complete pipeline including any training data and tools must be submitted as a zip archive containing all source files and a README describing how to build and run the project.

### Deliverable 2: Report (4000 words)

The report documents your implementation and provides the critical analysis through which learning outcomes are assessed. Structure it using the following sections.

**Introduction**
Briefly describe your game or interactive environment and the AI problem(s) you are addressing. State which techniques you have implemented and why you selected them.

**Design and Architecture**
Explain the overall structure of your AI system. Describe how the components fit together and how your design decisions follow from the constraints and goals of your chosen environment. Reference relevant literature to justify your design choices.

**Implementation**
Describe the key algorithmic choices you made during implementation. Where you deviated from a textbook approach, explain why. Include pseudocode or short code excerpts where they aid explanation; do not reproduce large blocks of source code.

**Evaluation**
Critically evaluate the AI behaviour you implemented. Discuss what works well, what fails under which conditions, and how your chosen approach compares to alternatives. Quantitative results (timing, path quality, decision accuracy) are expected where measurable.

**Reflection**
Reflect on the relationship between classical and modern AI approaches in your

implementation. Where could a machine learning technique replace or augment a classical one? What would be gained and what would be lost?

**References**
Full bibliography in a consistent citation format (Harvard preferred).

---

## Notes

The report is the primary evidence for your learning outcomes. Markers cannot infer understanding from code alone: every design decision, algorithmic choice, and evaluation finding must be articulated in the report. A functioning implementation with a weak report will not achieve a high mark; a strong report with a non-functional implementation will not pass.

The reference platform constraint is firm: your submission must build and run without modification on the platform specified in the lab setup (see Unit 01). Submissions that cannot be built will be marked on the report only.

Feedback will be provided in writing following marking. In-class tutor demonstrations of solution approaches will also be provided after the submission deadline.